

AI Game Agents: Minimax and Reinforcement Learning

CS7IS2 Artificial Intelligence - Assignment 3

Rakesh Lakshmanan

25337471

April 4, 2026

1 Introduction

The ability to play games is one of the earliest problems to be addressed in the field of artificial intelligence. Games like Tic Tac Toe (TTT) and Connect 4 are excellent candidates for evaluating decision-making algorithms, as they have very simple rules but can exhibit a wide variety of strategic complexity.

In this project, we develop and test the following two types of algorithms:

1. **Classical tree search:** Minimax with and without alpha-beta pruning, where all future game situations are calculated to make decisions.
2. **Reinforcement Learning (RL):** tabular Q-Learning and DQN, where decision-making is based on learning from experience rather than searching.

These four agents compete against a rule-based Default Opponent as well as each other in TTT and Connect 4.

2 Game Implementations

Each of them has an interface `BaseGame` with abstract methods for `reset`, `get_valid_moves`, `make_move`, `check_winner`, `clone`, and `get_state_key`. Players are coded as $+1$ and -1 , and empty cells are 0 .

2.1 Tic Tac Toe

The board size of TTT is 3×3 , and the moves are integers from 0 to 8 corresponding to the position on the board, which is represented by a NumPy array. Winning requires occupying any row, column, or diagonal line entirely. TTT is known to have at most 9 plies, about 5,478 states, and is fully solved: when both players play optimally, the result will be a draw.

2.2 Connect 4

Connect 4 is played on a board with dimensions 6×7 , where there is a gravity rule for the moves—the piece lands on the lowest row in the selected column. Column indices are 0–6 for each move. Winning involves aligning four pieces horizontally, vertically, or diagonally. It is estimated that the game tree of Connect 4 contains 4.5×10^{12} leaf nodes, and thus searching for the entire tree is impractical on current computers. Like TTT, Connect 4 is completely solved—the first player wins with optimal play, though the

3 Agent Implementations

3.1 Default Opponent

The Default Opponent is a deterministic agent which adheres to the following strict priority rules for selecting moves:

1. **Win:** Play a move that wins immediately.

2. **Block:** Block an opponent move that would win on the next turn.
3. **Preferred strategy:** In TTT, prioritize the center (cell 4) followed by corner cells (0, 2, 6, 8) followed by edge cells. In Connect 4, prioritize the central column (3) followed by its neighbors.
4. **Otherwise, choose a random move:** Make a uniformly random move if the above priorities do not apply.

Thus, the default opponent is guaranteed to be deterministic and strictly non-random.

3.2 Minimax

Minimax is a recursive algorithm which evaluates all potential future states of the game, assuming perfect play from both sides. The maximizing agent chooses the move with the best evaluation; the minimizing agent chooses the worst.

In TTT, the complete game tree is expanded to full depth (9 maximum). In Connect 4, the total number of potential moves grows too large (see Section 4.1), and so we use **depth-limited search** with `max_depth = 5` along with the heuristic evaluation function in Table 1.

Table 1: Heuristic evaluation weights for Connect 4.

Pattern	Weight
4-in-a-row (win)	+10,000
3-in-a-row with open space	+100
2-in-a-row with open spaces	+10
Centre column occupancy	+5
Opponent 3-in-a-row threat	-80

The agent also tracks the number of nodes explored during each search, enabling direct comparison with Alpha-Beta pruning.

3.3 Alpha-Beta Pruning

Alpha-Beta pruning extends Minimax by maintaining lower (α) and upper (β) bounds on the achievable evaluation at each node. Pruning occurs when the path will not result in an improvement over the existing value of the game. With ideal ordering, the effective branching factor goes from b to $\sqrt{b}$ [1].

The same heuristic function and maximum depth are employed for both algorithms, making sure that the two agents behave identically—the difference being that Alpha-Beta makes its choices more quickly than Minimax due to pruning.

3.4 Q-Learning

The Q-Learning algorithm stores a table containing Q-values for state-action pairs. Actions are determined using an ε -greedy policy where the agent will randomly choose one of the valid moves with probability ε or select the move with the largest Q-value otherwise.

After each transition (s, a, r, s') , the Q-value is updated:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (1)$$

The reward scheme is: +1 for a win, -1 for a loss, +0.3 for a draw, and 0 for intermediate moves. The hyperparameters used are listed in Table 2.

Table 2: Q-Learning hyperparameters.

Parameter	Value
Learning rate (α)	0.1
Discount factor (γ)	0.95
Initial ε	1.0
Minimum ε	0.01
ε decay rate	0.9995

In the case of TTT, the Q-Learning agent was trained on 30,000 episodes against the Default Opponent. In the case of Connect 4, the Q-Learning agent was trained on 50,000 episodes against the Random Agent (the reasons will be discussed further in Section 4.2).

3.5 Deep Q-Network (DQN)

The DQN agent uses a neural network to approximate $Q(s, a)$ values for all actions at once and hence learn generalisable actions. Some important elements of the DQN algorithm are the following:

- **Experience replay buffer**, size 10,000: Transitions are stored there and then used to form random mini-batches of 64 samples, eliminating temporal correlations.
- **Target network**: It is a copy of the main policy network that is updated once every 1,000 iterations.
- **Invalid action masking**: $-\infty$ value is assigned to Q values corresponding to actions impossible in the current state.

Architecture of the DQN for TTT: Input(9) $\rightarrow$ Dense(128) $\rightarrow$ ReLU $\rightarrow$ Dense(128) $\rightarrow$ ReLU $\rightarrow$ Output(9).

Connect 4 architecture: Input(42) $\rightarrow$ Dense(256) $\rightarrow$ ReLU $\rightarrow$ Dense(256) $\rightarrow$ ReLU $\rightarrow$ Dense(128) $\rightarrow$ ReLU $\rightarrow$ Output(7).

The DQN uses the same reward scheme as Q-Learning. For TTT, training ran for 15,000 episodes against the Default Opponent; for Connect 4, 25,000 episodes against a Random Agent.

4 Experimental Setup

- **Tournament games**: 100 per matchup, alternating which agent plays first each game.

- **Training episodes:** TTT Q-Learning: 50,000; TTT DQN: 20,000; Connect 4 Q-Learning: 100,000; Connect 4 DQN: 50,000.
- **Evaluation frequency:** Every 1,000 training episodes, the agent plays 100 evaluation games to measure current win rate.
- **Random seed:** 42.

4.1 Connect 4 Scalability Test

To empirically confirm the infeasibility of full-depth search in Connect 4, both Minimax and Alpha-Beta were run on an empty board for 30 minutes with no depth limit. The results are presented in Table 3.

Table 3: Connect 4 full-depth scalability test (30-minute budget, empty board).

Metric	Minimax	Alpha-Beta
Nodes explored in 30 min	28,262,928	3,872,513
Nodes per second	15,702	1,772
% of game tree explored	0.00021%	0.00003%
Estimated time for full tree	~27 years	~243 years
Timed out	Yes	Yes

In both cases, timeouts occurred before either agent had managed to explore a negligible number of the ~ 4.5 trillion possible nodes in the game tree. This clearly shows that exhaustive tree search is not computationally feasible, making the use of depth-limited tree search (`max_depth = 5`) along with heuristic evaluation functions justified. Under these conditions, each move is completed within less than 2 seconds while yielding meaningful moves.

Alpha-Beta Throughput Note: Although Alpha-Beta did manage to visit a fewer number of nodes (3.9 million versus 28.3 million), the per-node throughput for the former algorithm is smaller due to additional overheads related to the maintenance of α/β values on an empty and symmetric board.

4.2 RL Training Opponent for Connect 4

The state space for Connect 4 ($\sim 4.5 \times 10^{12}$ states) is simply too large to make convergence on Q-Learning against an adequate opponent feasible. Training on the Default Opponent results in an incredibly small number of positive rewards at the beginning of training due to losses in most games, resulting in very slow learning.

This issue was mitigated in training Connect 4 RL agents by utilizing the Random Agent to provide early reward signals and basic tactical learning before testing against better opponents.

5 Results

All tournaments consist of 100 games per matchup configuration. For each algorithm pair, games are played twice, once with each agent going first to isolate first-mover effects. This yields 8 matchups against the Default Opponent per game (4 algorithms $\times$ 2 move orders) and 12 head-to-head matchups per game (all ordered pairs of 4 algorithms, excluding self-play).

5.1 Algorithms vs Default Opponent in Tic Tac Toe

Figure 1 and Table 4 show each algorithm’s performance against the Default Opponent in TTT, broken down by move order.

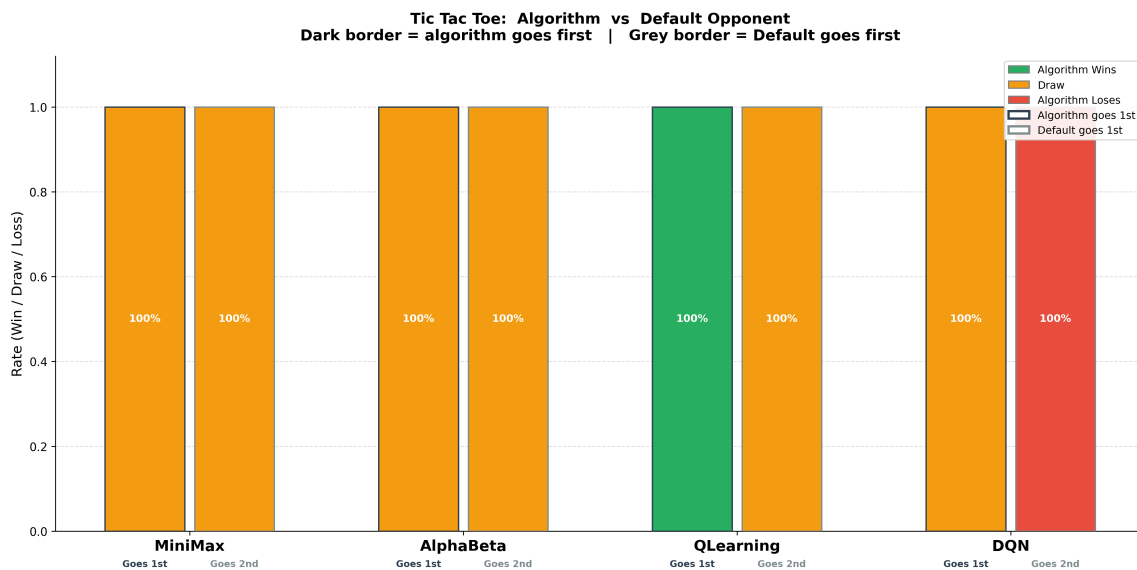


Figure 1: TTT: Algorithm performance vs Default Opponent (dark border = algorithm goes first; grey border = Default goes first).

Table 4: TTT results vs Default Opponent (100 games per configuration).

Algorithm	Goes	Win%	Draw%	Loss%	Avg. Length
Minimax	1st	0	100	0	9.0
Minimax	2nd	0	100	0	9.0
Alpha-Beta	1st	0	100	0	9.0
Alpha-Beta	2nd	0	100	0	9.0
Q-Learning	1st	100	0	0	7.0
Q-Learning	2nd	0	100	0	9.0
DQN	1st	0	100	0	9.0
DQN	2nd	0	0	100	7.0

The **minimax** and **alpha-beta** algorithm played each game independently of move ordering. This is the ideal result as Tic Tac Toe is a solved game where perfect strategy

from either side will guarantee a tie. The 100% tie ratio proves that the algorithm was correctly implemented.

Q-Learning performed best among all agents, securing a win rate of 100% when playing first and tying 100% when playing second, with zero losses in both scenarios. When the agent moves first, it capitalizes on the heuristic flaw of the default opponent: the default opponent follows an impossible to improve upon but slightly flawed heuristic priority list: win > block > center > corners > edges. After exploring all possible states through 50,000 games during training, Q-Learning developed a first mover strategy that the default opponent could not overcome. When the agent moves second, it perfectly plays defensively, always ensuring ties. The difference in the number of moves per game, which is lower when moving first (7.0 vs. 9.0), verifies that the agent wins in decisive fashion early.

DQN drew all games playing first and lost all games when playing second. The fact that DQN drew all games as the first mover but lost all games as the second mover shows that it learnt good offensive capabilities in terms of playing the first move to reach a draw. On the other hand, it did not learn how to defend well against the default opponent as seen from the 100% losses and short game lengths (7.0). These observations corroborate the instability of DQN during training (Section 5.5).

Key finding: Q-learning performs better than all other agents in TTT, including Minimax. This is surprising since Q-learning is an RL agent without any search mechanism but it exploited a vulnerability in the default opponent that the search-based algorithms were not able to do (Minimax guarantees draws as it assumes both sides have the best moves).

5.2 Algorithms vs Default Opponent — Connect 4

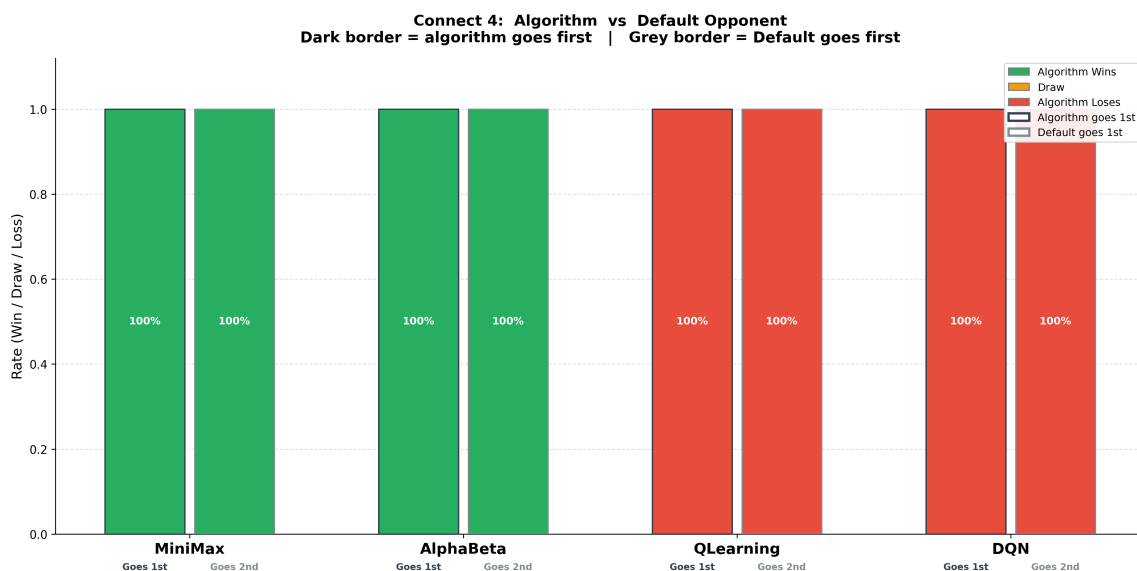


Figure 2: Connect 4: Algorithm performance vs Default Opponent.

Table 5: Connect 4 results vs Default Opponent (100 games per configuration).

Algorithm	Goes	Win%	Draw%	Loss%	Avg. Length
Minimax	1st	100	0	0	9.0
Minimax	2nd	100	0	0	20.0
Alpha-Beta	1st	100	0	0	9.0
Alpha-Beta	2nd	100	0	0	20.0
Q-Learning	1st	0	0	100	10.0
Q-Learning	2nd	0	0	100	17.0
DQN	1st	0	0	100	8.0
DQN	2nd	0	0	100	7.0

Minimax and Alpha-Beta achieved 100% victories in both starting positions, verifying that a depth-limited search approach (`max_depth = 5`) coupled with a heuristic evaluation function provides enough intelligence to outplay the rule-based Default Opponent. When playing first, the games tend to be short (avg. 9.0 moves), which stems from the large first-player advantage in Connect 4 and efficient identification of winning opportunities by the heuristic. If the agent plays second, the games tend to be much longer (avg. 20.0 moves), as the agent needs to disrupt the initiative of the opponent first before creating their own winning strategy.

Q-Learning lost every game in both starting positions. Although the agent achieved an impressive 80% victory rate while playing against the Random Agent in training (see Section 5.5), none of the strategies learned could be generalised against the Default Opponent. The Q-table, which is trained through random game playing, contains no Q-values for the states arising when playing against the opponent—in other words, the agent makes completely uninformed decisions.

DQN also failed at every game, although the average length of games was shorter (8.0 and 7.0). This suggests that the Default Opponent beat DQN faster than Q-Learning. This is in line with the poor training results observed for Connect 4 (Section 5.5).

Critical finding: The Connect 4 experiment highlights a clear demarcation between search-based and learning-based agents, with tree search outperforming even with a good heuristic. On the other hand, RL agents trained against weak opponents fail to generalise.

5.3 Head-to-Head Results — Tic Tac Toe

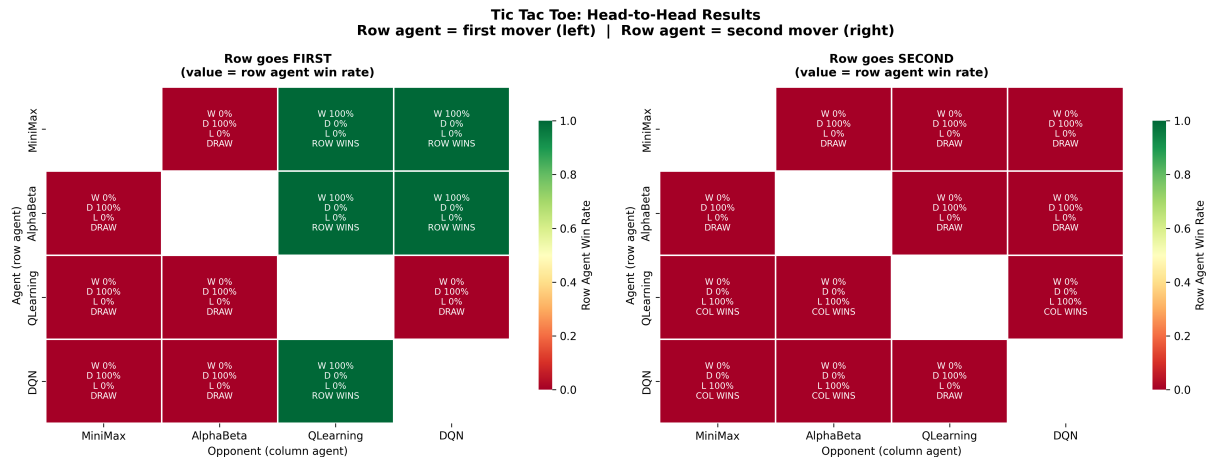


Figure 3: TTT: Head-to-head results. Left: row agent goes first. Right: row agent goes second. Each cell shows win/draw/loss percentages for the row agent.

Table 6: TTT head-to-head results (100 games per matchup). W/D/L from the first mover’s perspective.

First Mover	Second Mover	1st W%	D%	1st L%	Outcome
Minimax	AlphaBeta	0	100	0	Draw
Minimax	Q-Learning	100	0	0	1st wins
Minimax	DQN	100	0	0	1st wins
AlphaBeta	Minimax	0	100	0	Draw
AlphaBeta	Q-Learning	100	0	0	1st wins
AlphaBeta	DQN	100	0	0	1st wins
Q-Learning	Minimax	0	100	0	Draw
Q-Learning	AlphaBeta	0	100	0	Draw
Q-Learning	DQN	0	100	0	Draw
DQN	Minimax	0	100	0	Draw
DQN	AlphaBeta	0	100	0	Draw
DQN	Q-Learning	100	0	0	1st wins

The head-to-head matrix reveals several important patterns:

Minimax vs Alpha-Beta: Draws on all plays as expected. Since both programs use the same search procedure, they evaluate states equivalently and choose equivalent moves.

Minimax/Alpha-Beta vs Q-Learning: The tree-based agent wins 100% if it goes first. Q-learning achieves a draw rate of 100% when going first. This is a surprising result since Q-learning apparently learned the exact defensive strategy, that of never losing from either state but failed to learn how to beat Minimax as first mover since its attacks exploit some of the DefaultOpponent’s weaknesses and not optimal play. Indeed, Q-learning can not beat Minimax since the latter searches the entire response tree.

Minimax/Alpha-Beta vs DQN: The tree-searching agent wins 100% if it goes first. If DQN goes first, it manages to achieve a draw rate of 100%. DQN learned to defend but was unable to beat any opponent.

Q-Learning vs DQN: Q-Learning uses aspects from both positions, while DQN wins 100% of the time if moving first versus Q-Learning. This appears paradoxical—DQN wins as first mover against Q-Learning, yet Q-Learning has a better record. In truth, DQN has figured out a way to move first such that Q-Learning’s policy cannot interfere, whereas Q-Learning’s policy is generally more robust against any opponent.

First-mover advantage: There is a consistent trend—first movers will either win or tie each match; no first mover ever loses. This is because of the nature of TTT—first movers control the center and have an extra move.

Hierarchy: Minimax = Alpha-Beta > Q-Learning > DQN. Minimax and Alpha-Beta are optimally efficient tree searchers. Q-Learning, while the most effective RL agent, cannot lose to anything. DQN is least effective, losing to Q-Learning and tree searches when going second.

5.4 Head-to-Head Results — Connect 4

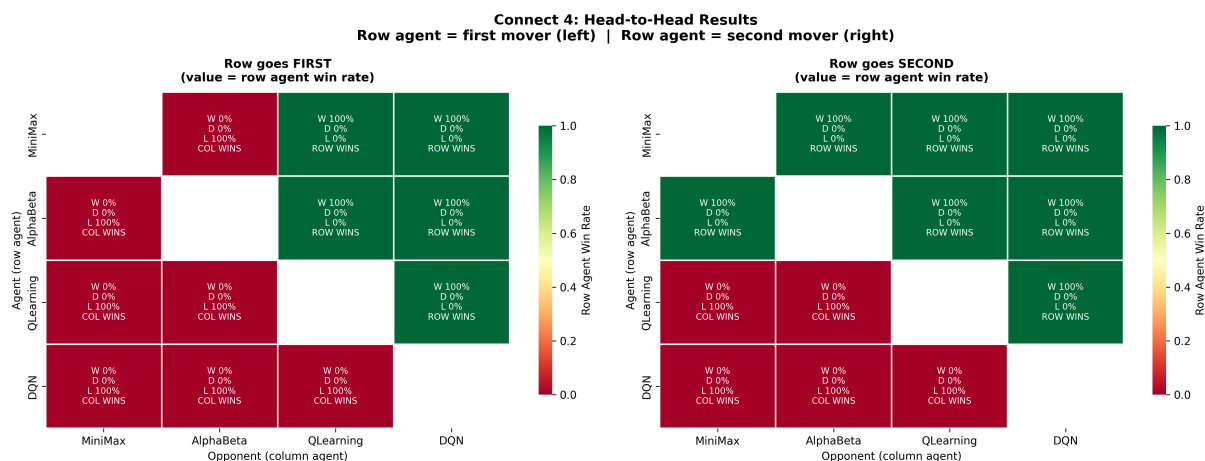


Figure 4: Connect 4: Head-to-head results. Left: row agent goes first. Right: row agent goes second.

Table 7: Connect 4 head-to-head results (100 games per matchup). W/D/L from the first mover’s perspective.

First Mover	Second Mover	1st W%	D%	1st L%	Outcome
Minimax	AlphaBeta	0	0	100	2nd wins
Minimax	Q-Learning	100	0	0	1st wins
Minimax	DQN	100	0	0	1st wins
AlphaBeta	Minimax	0	0	100	2nd wins
AlphaBeta	Q-Learning	100	0	0	1st wins
AlphaBeta	DQN	100	0	0	1st wins
Q-Learning	Minimax	0	0	100	2nd wins
Q-Learning	AlphaBeta	0	0	100	2nd wins
Q-Learning	DQN	100	0	0	1st wins
DQN	Minimax	0	0	100	2nd wins
DQN	AlphaBeta	0	0	100	2nd wins
DQN	Q-Learning	0	0	100	2nd wins

In Connect 4 matches between the two agents, the outcome is always a definitive 100% win for one agent with no ties.

Minimax vs Alpha-Beta: An unexpected outcome—in both cases, the *second* mover wins 100%. In games where Minimax goes first, Alpha-Beta (as the second mover) wins each game, and vice versa. Game lengths average 36.0 moves, showing difficult-to-resolve games. This is due to the fact that both agents employ depth-limited heuristics (`max_depth=5`). Since neither agent can see far enough ahead at depth 5 from the starting board, neither can guarantee a win, and both rely on the heuristic. The second mover has an advantage because the heuristic misvalues certain defensive plays, giving the reacting player more chances to secure their position. This is well-known artefact of depth-limited heuristics, using a biased heuristic against a player employing the same heuristic can be effective.

Tree search vs RL agents: Minimax and Alpha-Beta both win 100% of their matches against both RL agents in both directions. The games are short (7–14 moves), reflecting the skill disparity. The learned RL agent strategies, which have been trained against random opponents, are useless against tree search strategies.

Q-Learning vs DQN: Q-Learning wins 100% of the time when it starts, but also when DQN starts (DQN loses 100% in both situations). This makes Q-Learning better than DQN in Connect 4 hierarchy. The ~80% winning percentage of the tabular agent during training, compared to DQN’s ~75%, proved more effective tactics in the game.

Connect 4 without draws: Unlike TTT, there were no draws in any match-up because of larger board and more complicated tactics of the game, wins determine the outcome, especially in cases where one of the agents outperforms the other.

Connect 4 Hierarchy: Minimax = Alpha-Beta $\gg$ Q-Learning > DQN. The tree-based approaches belong to a league of their own, while among the reinforcement learning algorithms, Q-Learning takes the upper hand.

5.5 Learning Curves

Figure 5 provides a side-by-side overview of all four RL training runs.

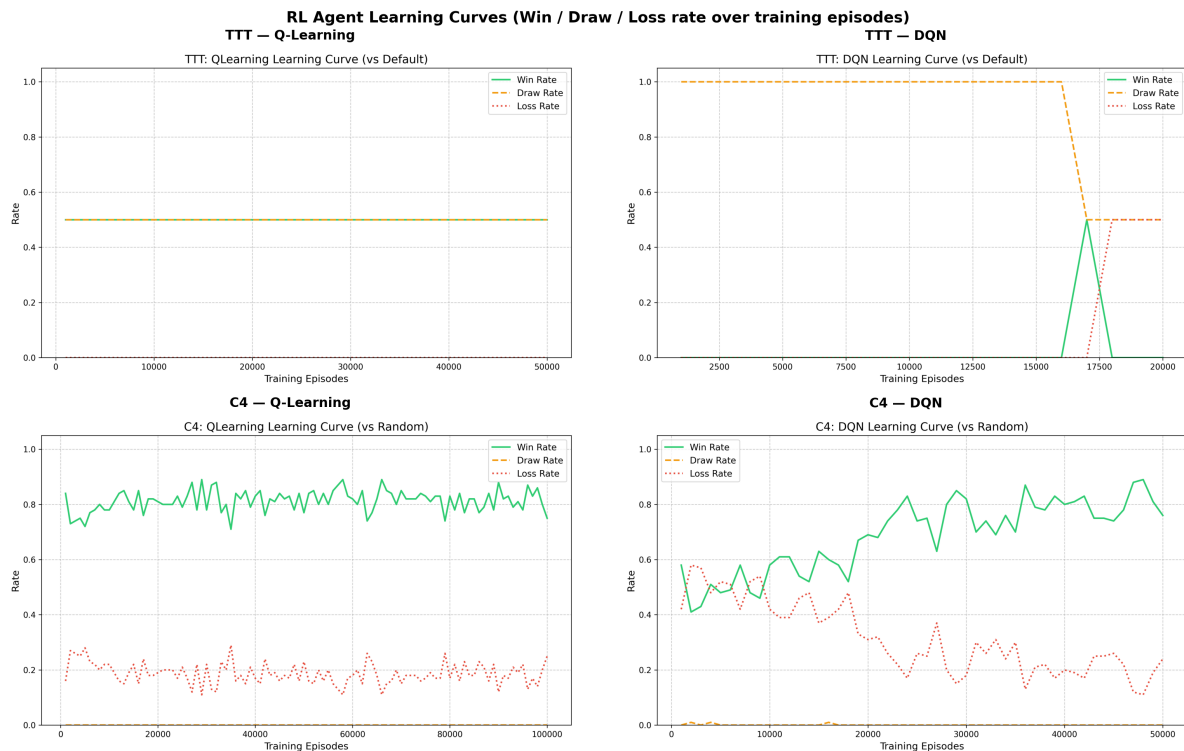


Figure 5: Summary of all RL agent learning curves. Top row: TTT. Bottom row: Connect 4. Left: Q-Learning. Right: DQN.

5.5.1 TTT: Q-Learning (vs Default Opponent, 50,000 episodes)

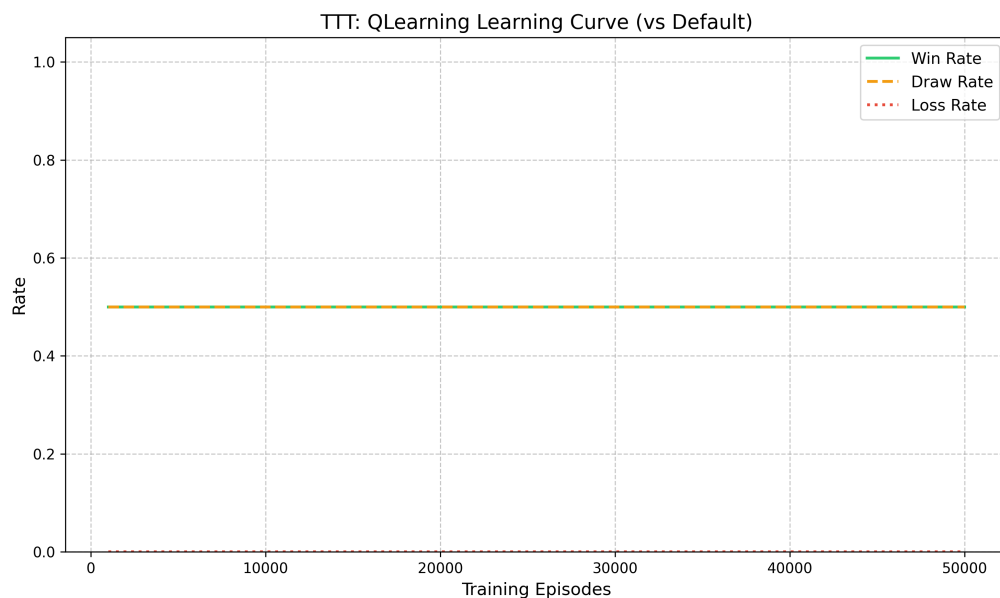


Figure 6: TTT: Q-Learning learning curve over 50,000 training episodes against the Default Opponent.

Figure 6 depicts that the Q-Learning curve for TTT exhibits *immediate* convergence in that the agent attains 50% win rate and 50% draw rate right at the start, and continues to achieve those exact percentages for all 50,000 evaluations, without a single loss.

It is easy to see why there is such rapid convergence here. First of all, there are less than 5,478 possible valid states, so each checkpoint after the earliest (after 1,000 episodes) covers a large part of the state space. The reason behind the perfect 50/50 split of wins and draws is the effect of the order of moves, where the agent wins 100% when playing first (due to the imperfectness of the Default Opponent's heuristic) and draws 100% while playing second (because it plays the game perfectly).

5.5.2 TTT: DQN (vs Default Opponent, 20,000 episodes)

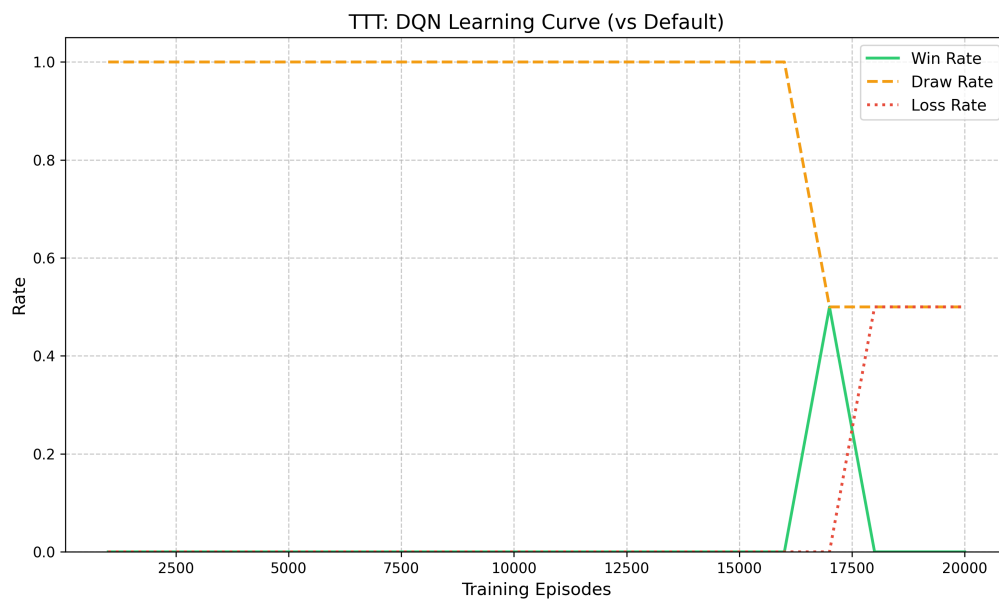


Figure 7: TTT: DQN learning curve over 20,000 training episodes against the Default Opponent.

The learning curve of the DQN model (Figure 7) presents an even more exciting story. In the initial phase up to ~15,000 episodes, the agent performs perfectly with 100% draws with zero wins and zero losses, meaning that it learns to avoid loss but never wins.

After episode 15,000-17,000, there is a sharp transition period where the draw rate drops significantly to about 50%, the win rate temporarily shoots up to ~50%, whereas the loss rate rises to about ~40%, followed by a steady state of roughly 0% wins, 50% draws, and 50% losses.

The drastic fall is a common occurrence caused by catastrophic forgetting, which refers to neural networks losing learned features from previous training. In the case of the decreasing ϵ , as exploration becomes minimal, the replay buffer fills primarily with recent data, causing the network to overfit on certain features at the cost of forgetting previously learned features such as defense against second mover positions.

5.5.3 Connect 4: Q-Learning (vs Random Agent, 100,000 episodes)

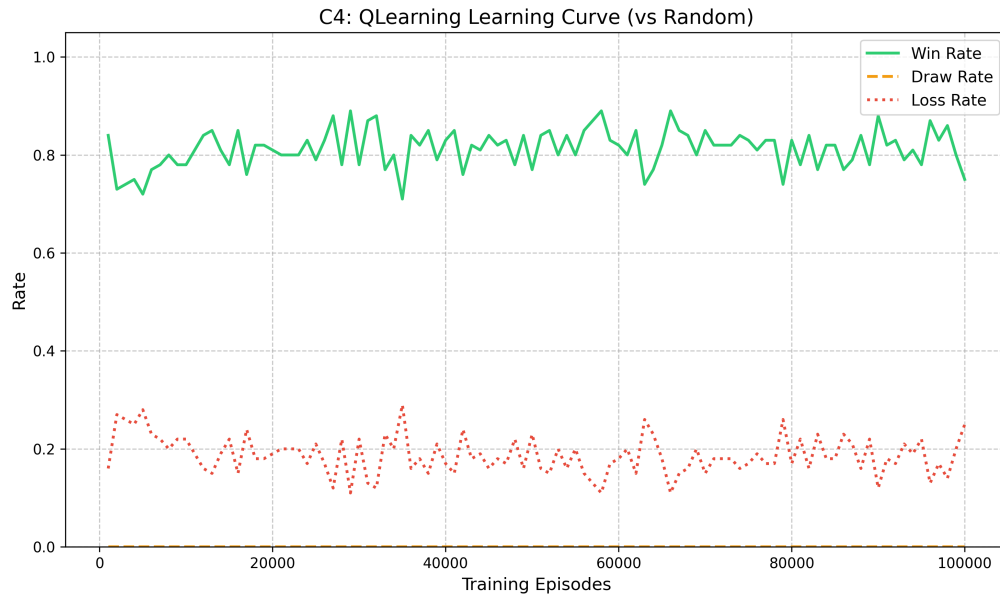


Figure 8: Connect 4: Q-Learning learning curve over 100,000 training episodes against a Random Agent.

In Connect 4, Q-learning converges to a win rate of $\sim 75 - 80\%$ after a few thousand episodes and sustains it during the entire training process; the loss rate converges to $\sim 15 - 20\%$, whereas the draw rate is consistently close to 0%.

The algorithm learns basic Connect 4 strategies fast, creating runs, controlling the center column, and taking advantage of any random errors. But the win rate converges to $\sim 80\%$, not to 100%, as even a Random Agent may sometimes make random blocking moves. The persistent loss rate of $\sim 20\%$ stems from the intrinsic limitation of tabular RL in Connect 4's large state space of $\sim 10^{12}$, it is impossible to learn Q-values in states that never occur.

5.5.4 Connect 4: DQN (vs Random Agent, 50,000 episodes)

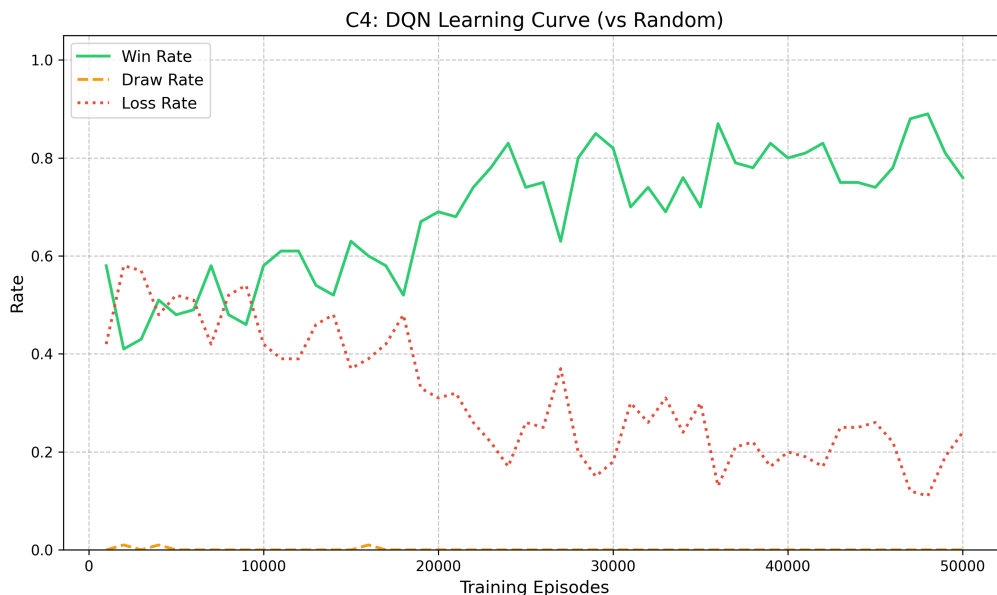


Figure 9: Connect 4: DQN learning curve over 50,000 training episodes against a Random Agent.

DQN learning for Connect 4 (see Figure 9) actually improves. From initial performance around 50% wins and 50% losses, performance eventually improves such that the win rate climbs to around 70% after 20,000 episodes and hits a peak of 80-89% by episode 40,000-50,000, with the loss rate around 10-25% by then.

In contrast to Q-Learning, which takes only one episode to achieve near-peak performance of 80%, the DQN algorithm is both slower (needing around 20,000 episodes to get to 70%), and more inconsistent (higher variance in evaluations). This is expected, since the neural network must learn its own representation of features, whereas the Q-table learns the values directly.

5.6 Nodes Explored: Minimax vs Alpha-Beta

For Connect 4 at depth 5, the vanilla minimax algorithm expanded 19,607 nodes against 2,852 from Alpha-Beta, a speedup factor of $6.9\times$. Although a large amount of computation is saved, the same evaluations are made by both because Alpha-Beta is a pure optimiser, and does not change the minimax value of any node.

In the full-depth scalability test (Table 3), the difference was even more dramatic: Minimax explored 28.3M nodes vs Alpha-Beta's 3.9M over 30 minutes, a $7.3\times$ reduction.

5.7 Overall Comparison

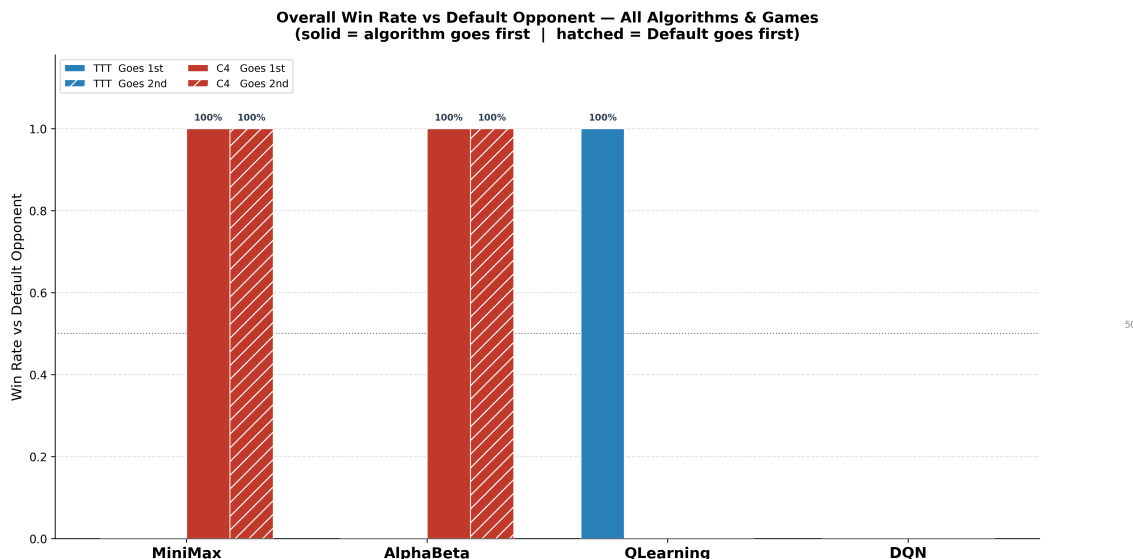


Figure 10: Overall win rate vs Default Opponent across both games (solid = algorithm goes first; hatched = Default goes first).

Figure 10 summarises the overall picture across both games:

- **Minimax and Alpha-Beta** achieve 100% win rate in Connect 4 (both move orders) but 0% win rate in TTT (drawing instead, which is optimal). Their TTT bars are absent from the win-rate chart because draws are not counted as wins.
- **Q-Learning** is the only agent to achieve wins in TTT (100% when going first), but scores 0% in Connect 4.
- **DQN** achieves 0% win rate across both games and both move orders—it only draws (TTT 1st) or loses.

No single algorithm dominates across all conditions. Minimax/Alpha-Beta are the strongest in Connect 4 but cannot exploit the Default Opponent in TTT (by design—they assume optimal counter-play). Q-Learning is the strongest in TTT but fails in Connect 4. DQN is consistently the weakest.

6 Analysis and Discussion

6.1 Minimax vs Alpha-Beta

Minimax and Alpha-Beta generate the same results for all tests except when playing against each other in Connect 4, where the second move wins the games. The reason for this oddity is the same search depth and heuristic function, which favors the reactive player at depth 5 due to its bias. With more search depth and a better heuristic, such behavior will probably not occur again.

6.2 Q-Learning: The Surprise Performer in TTT

The most surprising finding is the superiority of Q-Learning in the game of TTT. The tabular learning agent reached convergence very quickly, produced outstanding performance on TTT:

- Perfect win rate as first mover against Default Opponent (outperforming minimax's win / draw ratio of 0% win / 100% draw)
- 0% loss ratio in all settings (Default Opponent and head-to-head agents included)
- The only algorithm to capitalize on Default Opponent heuristic weakness

This finding draws attention to an important theoretical distinction: while **Minimax assumes that both players are playing optimally**, thus never trying to exploit their opponent's weaknesses, it instead guarantees the maximum possible payoff - a tie in TTT case. Q-Learning algorithm was trained on one specific opponent, so it took advantage of his particular weaknesses.

6.3 DQN: Training Instability

DQN was the worst-performing agent in both games, despite the theoretical ability to tackle large state spaces via function approximation. Two problems stood out:

1. **Catastrophic forgetting in TTT:** In spite of a stable 100% draw rate for 15,000 episodes, the policy deteriorated, it lost its defensive mechanism due to the shifting nature of the replay buffer. Possible solutions include prioritised experience replay, larger buffer size, or snapshotting the policy at regular intervals.
2. **Convergence speed in Connect 4:** It took DQN ~20,000 episodes to achieve a win rate of 70% against random agents, while Q-Learning got up to 80% in just 1,000 episodes. The additional cost incurred by the neural network for learning the feature space representation from scratch is much higher than simple tabular memory.

This is not surprising considering the limitations of DQN training in small- to-medium sized state spaces, where tabular methods will always outperform function approximation since the state space is small

6.4 The Role of Training Opponent

Another theme running across all the experiments is the necessity of the training opponent for RL agents. Q-Learning trained on the Default Opponent in TTT learned how to take advantage of the Default Opponent's weaknesses. Both RL agents trained on Random in Connect 4 learned simple strategies but were unable to deal with more complex opponents.

Therefore, a possible way to proceed is via **curriculum learning**, where RL agents begin with training on a Random Agent to learn simple strategies, followed by increasingly challenging opponents (Default, then Self-play).

6.5 Comparison Across Games

Table 8 summarises each agent's performance across all experimental conditions.

Table 8: Summary of agent performance across all conditions.

	TTT vs De-fault	TTT H2H	C4 vs Default	C4 H2H
Minimax	0W / 100D / 0L (both orders)	Wins 1st vs RL; Draws otherwise	100W / 0D / 0L (both orders)	Wins vs RL; Loses to AB (2nd wins)
Alpha-Beta	0W / 100D / 0L (both orders)	Identical to Minimax	100W / 0D / 0L (both orders)	Identical to Minimax
Q-Learning	100W 1st; 100D 2nd	Never loses; Draws vs all except wins as 1st vs Default	0W / 0D / 100L (both orders)	Beats DQN (1st); Loses to tree search
DQN	100D 1st; 100L 2nd	Draws 1st; Loses 2nd (except beats QL as 1st)	0W / 0D / 100L (both orders)	Loses to every- one

TTT (5,478 states) has a sufficiently small state space such that all agents are capable of performing at an adequate level, while RL algorithms actually perform better than tree search against non-optimal opponents. In Connect 4 (1012 states), we begin to see the boundaries of what RL can accomplish. Q-Learning cannot possibly learn the entire state space, and DQN needs much more training and architecture fine-tuning than was done here.

6.6 Trade-offs Summary

Table 9: Agent trade-offs.

Agent	Strengths	Weaknesses	Best at	Worst at
Minimax	Optimal play; no training; deterministic	Cannot exploit imperfect opponents; exponential time	C4 vs De-fault	TTT win rate
Alpha-Beta	Same as Minimax, 7× faster	Same limitations	C4 vs De-fault	TTT win rate
Q-Learning	Exploits opponent weaknesses; fast convergence (small games)	Huge table; fails in large state spaces	TTT vs Default	C4 vs De-fault
DQN	Generalises via neural net; handles large state spaces (in theory)	Unstable training; catastrophic forgetting; slow convergence	C4 learning curve	All eval matchups

7 Conclusions

This assignment evaluates four algorithms for gaming AI: Minimax, Alpha-Beta pruning, tabular Q-learning, and Deep Q-networks. They are tested for the games of Tic-Tac-Toe and Connect Four.

Important findings includes:

1. **No single algorithm dominates all settings.** Minimax/Alpha-Beta are strongest in Connect 4 (100% win rate), while Q-Learning is strongest in TTT (100% win rate as first mover, 0% losses overall). Algorithm selection depends on the game's state space and the opponent's characteristics.
2. **Tree search excels in larger games.** Depth-limited Minimax/Alpha-Beta won every Connect 4 game against all opponents. Alpha-Beta pruning provides a 6.9–7.3× efficiency gain with no loss in decision quality, enabling deeper search within the same time budget.
3. **Q-learning may perform better than minimax when playing against imperfect opponents.** As minimax always assumes an optimal move from the adversary, it plays very cautiously and will always end up drawing the game. The Q-learning algorithm, however, having learned from the particular adversary, found its weakness and ended up winning the game.
4. **DQN suffers from training instability.** Catastrophic forgetting in TTT and slow convergence in Connect 4 made DQN the weakest agent overall. While DQN's ability to generalise is theoretically superior to tabular Q-Learning, realising this advantage requires careful architecture design, hyperparameter tuning, and training stabilisation techniques beyond the scope of this work.
5. **First-mover advantage is significant.** In TTT, the first mover never lost in any matchup. In Connect 4, first-mover advantage was so strong that tree-search agents won in just 9 moves when going first (vs 20 when going second).
6. **Training opponent choice critically affects RL generalisation.** RL agents trained against a Random Agent learned basic tactics but failed against stronger opponents. A curriculum learning approach—gradually increasing opponent strength—would likely improve generalisation.

Future work could explore self-play training as in AlphaZero [4], Monte Carlo Tree Search (MCTS) as a middle ground between pure search and pure learning, curriculum learning with progressively stronger opponents, and more extensive hyperparameter tuning for DQN including prioritised experience replay, dueling architectures, and longer training runs.

References

- [1] Russell, S. and Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson.
- [2] Sutton, R.S. and Barto, A.G. (2018). *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press.

- [3] Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518, 529–533.
- [4] Silver, D., Schrittwieser, J., Simonyan, K., et al. (2017). Mastering the game of Go without human knowledge. *Nature*, 550, 354–359.
- [5] Allis, V. (1988). A knowledge-based approach of Connect-Four. *ICCA Journal*, 11(4), 165–175.

A Appendix 1 – Minimax TTT

Listing 1: Minimax algorithm for Tic Tac Toe (with optional Alpha-Beta pruning)

```

1 def minimax_ttt(game, depth, is_max, player, opp, use_ab, alpha, beta
  ):
2     global nodes_visited
3     nodes_visited += 1
4
5     w = game.check_winner()
6     if w == player:
7         return 10 - depth, None
8     if w == opp:
9         return depth - 10, None
10    if game.is_draw():
11        return 0, None
12
13    moves = game.get_available_moves()
14    best_move = None
15
16    if is_max:
17        val = -math.inf
18        for m in moves:
19            game.make_move(m, player)
20            score, _ = minimax_ttt(game, depth+1, False, player, opp,
21                                use_ab, alpha, beta)
22            game.undo_move(m)
23            if score > val:
24                val = score
25                best_move = m
26            if use_ab:
27                alpha = max(alpha, val)
28                if beta <= alpha:
29                    break
30        return val, best_move
31    else:
32        val = math.inf
33        for m in moves:
34            game.make_move(m, opp)
35            score, _ = minimax_ttt(game, depth+1, True, player, opp,
36                                use_ab, alpha, beta)
37            game.undo_move(m)
38            if score < val:
39                val = score
40                best_move = m
41            if use_ab:
42                beta = min(beta, val)
43                if beta <= alpha:

```

```

44         break
45     return val, best_move

```

B Appendix 2 – Minimax TTT with Alpha-Beta Pruning

Alpha-Beta pruning is integrated into the same function shown in Appendix A by passing `use_ab=True`. When enabled, the algorithm maintains α (lower bound) and β (upper bound) at each node and prunes subtrees via the `if beta <= alpha: break` condition. This does not alter the minimax value returned—it only reduces the number of nodes explored.

C Appendix 3 – Minimax Connect 4

Listing 2: Minimax agent for Connect 4 with depth-limited heuristic search

```

1  class MiniMaxAgent(BaseAgent):
2      def __init__(self, player, name="MiniMax", max_depth=None):
3          super().__init__(player, name)
4          self.max_depth = max_depth
5          self.nodes_explored = 0
6
7      def get_move(self, game):
8          self.nodes_explored = 0
9          valid_moves = game.get_valid_moves()
10         best_move = valid_moves[0]
11         best_score = float('-inf')
12
13         for move in valid_moves:
14             g = game.clone()
15             g.make_move(move, self.player)
16             score = self._minimax(g, 1, False)
17             if score > best_score:
18                 best_score = score
19                 best_move = move
20         return best_move
21
22     def _minimax(self, game, depth, is_maximizing):
23         self.nodes_explored += 1
24         winner = game.check_winner()
25         if winner is not None:
26             if winner == self.player:
27                 return 10000 - depth
28             elif winner == -self.player:
29                 return depth - 10000
30             else:
31                 return 0
32
33         if self.max_depth is not None and depth >= self.max_depth:
34             return self._evaluate(game)
35
36         valid_moves = game.get_valid_moves()
37         if not valid_moves:

```

```
38         return 0
39
40     current_player = self.player if is_maximizing else -self.
        player
41
42     if is_maximizing:
43         best = float('-inf')
44         for move in valid_moves:
45             g = game.clone()
46             g.make_move(move, current_player)
47             score = self._minimax(g, depth + 1, False)
48             best = max(best, score)
49         return best
50     else:
51         best = float('inf')
52         for move in valid_moves:
53             g = game.clone()
54             g.make_move(move, current_player)
55             score = self._minimax(g, depth + 1, True)
56             best = min(best, score)
57         return best
58
59     def _connect4_heuristic(self, game):
60         score = 0
61         b = game.board
62         rows, cols = b.shape
63
64     def score_window(window, player):
65         opp = -player
66         s = 0
67         pc = list(window).count(player)
68         ec = list(window).count(0)
69         oc = list(window).count(opp)
70         if pc == 4:
71             s += 10000
72         elif pc == 3 and ec == 1:
73             s += 100
74         elif pc == 2 and ec == 2:
75             s += 10
76         if oc == 3 and ec == 1:
77             s -= 80
78         return s
79
80     # Centre column preference
81     center_col = list(b[:, cols // 2])
82     score += center_col.count(self.player) * 5
83     score -= center_col.count(-self.player) * 5
84
85     # Horizontal windows
86     for r in range(rows):
87         for c in range(cols - 3):
88             window = b[r, c:c+4]
89             score += score_window(window, self.player)
90
91     # Vertical windows
92     for r in range(rows - 3):
93         for c in range(cols):
94             window = b[r:r+4, c]
```

```

95         score += score_window(window, self.player)
96
97     # Positive diagonal
98     for r in range(rows - 3):
99         for c in range(cols - 3):
100             window = [b[r+i, c+i] for i in range(4)]
101             score += score_window(window, self.player)
102
103     # Negative diagonal
104     for r in range(rows - 3):
105         for c in range(3, cols):
106             window = [b[r+i, c-i] for i in range(4)]
107             score += score_window(window, self.player)
108
109     return score

```

D Appendix 4 – Minimax Connect 4 with Alpha-Beta Pruning

Listing 3: Alpha-Beta pruning agent for Connect 4

```

1  class AlphaBetaAgent(BaseAgent):
2      def __init__(self, player, name="AlphaBeta", max_depth=None):
3          super().__init__(player, name)
4          self.max_depth = max_depth
5          self.nodes_explored = 0
6
7      def get_move(self, game):
8          self.nodes_explored = 0
9          valid_moves = game.get_valid_moves()
10         best_move = valid_moves[0]
11         best_score = float('-inf')
12         alpha = float('-inf')
13         beta = float('inf')
14
15         for move in valid_moves:
16             g = game.clone()
17             g.make_move(move, self.player)
18             score = self._alphabeta(g, 1, False, alpha, beta)
19             if score > best_score:
20                 best_score = score
21                 best_move = move
22             alpha = max(alpha, score)
23         return best_move
24
25     def _alphabeta(self, game, depth, is_maximizing, alpha, beta):
26         self.nodes_explored += 1
27         winner = game.check_winner()
28         if winner is not None:
29             if winner == self.player:
30                 return 10000 - depth
31             elif winner == -self.player:
32                 return depth - 10000
33         else:
34             return 0
35

```

```

36         if self.max_depth is not None and depth >= self.max_depth:
37             return self._evaluate(game)
38
39         valid_moves = game.get_valid_moves()
40         if not valid_moves:
41             return 0
42
43         current_player = self.player if is_maximizing \
44                             else -self.player
45
46         if is_maximizing:
47             best = float('-inf')
48             for move in valid_moves:
49                 g = game.clone()
50                 g.make_move(move, current_player)
51                 score = self._alphabeta(g, depth+1, False,
52                                         alpha, beta)
53                 best = max(best, score)
54                 alpha = max(alpha, best)
55                 if beta <= alpha:
56                     break # Beta cutoff
57             return best
58         else:
59             best = float('inf')
60             for move in valid_moves:
61                 g = game.clone()
62                 g.make_move(move, current_player)
63                 score = self._alphabeta(g, depth+1, True,
64                                         alpha, beta)
65                 best = min(best, score)
66                 beta = min(beta, best)
67                 if beta <= alpha:
68                     break # Alpha cutoff
69             return best

```

The heuristic evaluation function is identical to Appendix C and is omitted for brevity.

E Appendix 5 – Q-Learning Agent

Listing 4: Tabular Q-Learning agent with epsilon-greedy exploration

```

1 class QLearningAgent(BaseAgent):
2     def __init__(self, player, name="QLearning",
3                 alpha=0.1, gamma=0.95,
4                 epsilon=1.0, epsilon_min=0.01,
5                 epsilon_decay=0.9995, seed=None):
6         super().__init__(player, name)
7         self.alpha = alpha
8         self.gamma = gamma
9         self.epsilon = epsilon
10        self.epsilon_min = epsilon_min
11        self.epsilon_decay = epsilon_decay
12        self.q_table = {}
13        if seed is not None:
14            random.seed(seed)
15
16    def _get_q(self, state_key, action):

```

```

17         return self.q_table.get((state_key, action), 0.0)
18
19     def state_key(self, game):
20         """Normalise board so agent's pieces always appear as +1."""
21         if self.player == 1:
22             return game.get_state_key()
23         return str(tuple((-game.board).flatten()))
24
25     def get_move(self, game):
26         valid_moves = game.get_valid_moves()
27         if not valid_moves:
28             return None
29         # Epsilon-greedy action selection
30         if random.random() < self.epsilon:
31             return random.choice(valid_moves)
32         sk = self.state_key(game)
33         q_values = {a: self._get_q(sk, a) for a in valid_moves}
34         return max(q_values, key=q_values.get)
35
36     def learn(self, state, action, reward, next_state,
37               done, valid_next_moves):
38         current_q = self._get_q(state, action)
39         if done or not valid_next_moves:
40             target = reward
41         else:
42             max_next_q = max(self._get_q(next_state, a)
43                             for a in valid_next_moves)
44             target = reward + self.gamma * max_next_q
45         self.q_table[(state, action)] = (
46             current_q + self.alpha * (target - current_q)
47         )
48
49     def decay_epsilon(self):
50         self.epsilon = max(self.epsilon_min,
51                             self.epsilon * self.epsilon_decay)
52
53     def save(self, filepath):
54         os.makedirs(os.path.dirname(filepath) or '.',
55                     exist_ok=True)
56         with open(filepath, 'wb') as f:
57             pickle.dump({'q_table': self.q_table,
58                         'epsilon': self.epsilon}, f)
59
60     def load(self, filepath):
61         with open(filepath, 'rb') as f:
62             data = pickle.load(f)
63             self.q_table = data['q_table']
64             self.epsilon = data.get('epsilon', self.epsilon_min)

```

F Appendix 6 – Deep Q-Network (DQN) Agent

Listing 5: DQN agent with experience replay and target network

```

1 class DQNNetwork(nn.Module):
2     def __init__(self, input_size, output_size):
3         super().__init__()

```

```

4         if input_size == 9: # TTT
5             self.net = nn.Sequential(
6                 nn.Linear(input_size, 128), nn.ReLU(),
7                 nn.Linear(128, 128), nn.ReLU(),
8                 nn.Linear(128, output_size),
9             )
10        else: # Connect 4
11            self.net = nn.Sequential(
12                nn.Linear(input_size, 256), nn.ReLU(),
13                nn.Linear(256, 256), nn.ReLU(),
14                nn.Linear(256, 128), nn.ReLU(),
15                nn.Linear(128, output_size),
16            )
17
18        def forward(self, x):
19            return self.net(x)
20
21
22    class DQNAgent(BaseAgent):
23        def __init__(self, player, name="DQN",
24                      input_size=9, output_size=9,
25                      alpha=0.001, gamma=0.95,
26                      epsilon=1.0, epsilon_min=0.01,
27                      epsilon_decay=0.9995, batch_size=64,
28                      replay_buffer_size=10000,
29                      target_update=1000, seed=None):
30            super().__init__(player, name)
31            self.input_size = input_size
32            self.output_size = output_size
33            self.gamma = gamma
34            self.epsilon = epsilon
35            self.epsilon_min = epsilon_min
36            self.epsilon_decay = epsilon_decay
37            self.batch_size = batch_size
38            self.target_update = target_update
39            self.steps = 0
40
41            if seed is not None:
42                torch.manual_seed(seed)
43
44            self.device = torch.device('cpu')
45            self.policy_net = DQNNetwork(
46                input_size, output_size).to(self.device)
47            self.target_net = DQNNetwork(
48                input_size, output_size).to(self.device)
49            self.target_net.load_state_dict(
50                self.policy_net.state_dict())
51            self.target_net.eval()
52
53            self.optimizer = optim.Adam(
54                self.policy_net.parameters(), lr=alpha)
55            self.replay_buffer = deque(
56                maxlen=replay_buffer_size)
57
58        def _normalize(self, board):
59            """Agent's pieces always appear as +1."""
60            return board * self.player
61

```

```

62     def get_move(self, game):
63         valid_moves = game.get_valid_moves()
64         if not valid_moves:
65             return None
66         if random.random() < self.epsilon:
67             return random.choice(valid_moves)
68         state = torch.FloatTensor(
69             self._normalize(game.board).flatten()
70         ).unsqueeze(0).to(self.device)
71         with torch.no_grad():
72             q_values = self.policy_net(state).squeeze()
73         # Mask invalid actions
74         mask = torch.full(
75             (self.output_size,), float('-inf'))
76         for m in valid_moves:
77             mask[m] = 0
78         q_values = q_values + mask
79         return q_values.argmax().item()
80
81     def store_transition(self, state, action, reward,
82                        next_state, done):
83         self.replay_buffer.append((
84             self._normalize(state).flatten().copy(),
85             action, reward,
86             self._normalize(next_state).flatten().copy(),
87             done,
88         ))
89
90     def train_step(self):
91         if len(self.replay_buffer) < self.batch_size:
92             return 0.0
93         batch = random.sample(
94             self.replay_buffer, self.batch_size)
95         states, actions, rewards, next_states, dones = \
96             zip(*batch)
97
98         states = torch.FloatTensor(
99             np.array(states)).to(self.device)
100        actions = torch.LongTensor(
101            actions).to(self.device)
102        rewards = torch.FloatTensor(
103            rewards).to(self.device)
104        next_states = torch.FloatTensor(
105            np.array(next_states)).to(self.device)
106        dones = torch.FloatTensor(
107            dones).to(self.device)
108
109        current_q = self.policy_net(states).gather(
110            1, actions.unsqueeze(1)).squeeze(1)
111        with torch.no_grad():
112            next_q = self.target_net(
113                next_states).max(1)[0]
114        target_q = rewards + \
115            (1 - dones) * self.gamma * next_q
116
117        loss = nn.MSELoss()(current_q, target_q)
118        self.optimizer.zero_grad()
119        loss.backward()

```

```

120         self.optimizer.step()
121
122         self.steps += 1
123         if self.steps % self.target_update == 0:
124             self.target_net.load_state_dict(
125                 self.policy_net.state_dict())
126         return loss.item()
127
128     def decay_epsilon(self):
129         self.epsilon = max(self.epsilon_min,
130                             self.epsilon * self.epsilon_decay)

```

G Appendix 7 – Default Opponent

Listing 6: Rule-based Default Opponent (win > block > strategic > random)

```

1  class DefaultOpponent(BaseAgent):
2      def __init__(self, player, name="DefaultOpponent",
3                  seed=None):
4          super().__init__(player, name)
5          if seed is not None:
6              random.seed(seed)
7
8      def get_move(self, game):
9          valid_moves = game.get_valid_moves()
10         if not valid_moves:
11             return None
12
13         # Priority 1: Win if possible
14         for move in valid_moves:
15             g = game.clone()
16             g.make_move(move, self.player)
17             if g.check_winner() == self.player:
18                 return move
19
20         # Priority 2: Block opponent win
21         opponent = -self.player
22         for move in valid_moves:
23             g = game.clone()
24             g.make_move(move, opponent)
25             if g.check_winner() == opponent:
26                 return move
27
28         # Priority 3: Strategic position preference
29         from games.tic_tac_toe import TicTacToe
30         from games.connect4 import Connect4
31         if isinstance(game, TicTacToe):
32             # Centre > corners > edges
33             preferred = [4, 0, 2, 6, 8, 1, 3, 5, 7]
34             for move in preferred:
35                 if move in valid_moves:
36                     return move
37         elif isinstance(game, Connect4):
38             # Centre column outward
39             preferred = [3, 2, 4, 1, 5, 0, 6]
40             for move in preferred:

```

```

41         if move in valid_moves:
42             return move
43
44     # Priority 4: Random fallback
45     return random.choice(valid_moves)

```

H Appendix 8 – Training Loop

Listing 7: RL agent training loop with periodic evaluation

```

1  def train_rl_agent(agent, opponent, game_class,
2                      num_episodes=50000, eval_every=1000,
3                      eval_games=100, seed=42):
4      random.seed(seed)
5      np.random.seed(seed)
6      training_history = []
7      is_dqn = hasattr(agent, 'store_transition')
8
9      for episode in tqdm(range(1, num_episodes + 1),
10                          desc=f"Training {agent.name}"):
11          game = game_class()
12          game.reset()
13
14          # Alternate first mover each episode
15          if episode % 2 == 0:
16              agent.player = 1
17              opponent.player = -1
18          else:
19              agent.player = -1
20              opponent.player = 1
21
22          current_player_id = 1
23          agent_board_before = None
24          prev_agent_sk = None
25          prev_agent_action = None
26
27          while True:
28              if current_player_id == agent.player:
29                  agent_board_before = game.board.copy()
30                  if not is_dqn:
31                      prev_agent_sk = agent.state_key(game)
32
33                  action = agent.get_move(game)
34                  next_state, reward, done, info = \
35                      game.make_move(action, agent.player)
36                  prev_agent_action = action
37
38                  if done:
39                      if is_dqn:
40                          agent.store_transition(
41                              agent_board_before, action,
42                              reward, next_state, done)
43                          agent.train_step()
44                      else:
45                          agent.learn(
46                              prev_agent_sk, action,

```

```

47         reward, prev_agent_sk,
48         True, [])
49     break
50 else:
51     opp_action = opponent.get_move(game)
52     next_state, _, done, info = \
53         game.make_move(
54             opp_action, opponent.player)
55
56     if done:
57         winner = info.get('winner')
58         agent_reward = (
59             1.0 if winner == agent.player
60             else 0.3 if winner == 0
61             else -1.0)
62     else:
63         agent_reward = 0.0
64
65     if agent_board_before is not None:
66         if is_dqn:
67             agent.store_transition(
68                 agent_board_before,
69                 prev_agent_action,
70                 agent_reward,
71                 next_state, done)
72             agent.train_step()
73         else:
74             if done:
75                 agent.learn(
76                     prev_agent_sk,
77                     prev_agent_action,
78                     agent_reward,
79                     prev_agent_sk, True, [])
80             else:
81                 next_sk = \
82                     agent.state_key(game)
83                 agent.learn(
84                     prev_agent_sk,
85                     prev_agent_action,
86                     0.0, next_sk, False,
87                     game.get_valid_moves())
88             agent_board_before = None
89
90     if done:
91         break
92
93     current_player_id = -current_player_id
94     agent.decay_epsilon()
95
96     # Periodic evaluation
97     if episode % eval_every == 0:
98         wins = draws = losses = 0
99         saved_eps = agent.epsilon
100         agent.epsilon = 0.0
101
102     for g_idx in range(eval_games):
103         eval_game = game_class()
104         eval_game.reset()

```

```

105         if g_idx % 2 == 0:
106             agent.player = 1
107             opponent.player = -1
108         else:
109             agent.player = -1
110             opponent.player = 1
111
112         cur = 1
113         while True:
114             if cur == agent.player:
115                 m = agent.get_move(eval_game)
116             else:
117                 m = opponent.get_move(eval_game)
118             _, _, d, info = \
119                 eval_game.make_move(m, cur)
120             if d:
121                 w = info.get('winner')
122                 if w == agent.player:
123                     wins += 1
124                 elif w == 0:
125                     draws += 1
126                 else:
127                     losses += 1
128                 break
129             cur = -cur
130
131         agent.epsilon = saved_eps
132         training_history.append({
133             'episode': episode,
134             'win_rate': wins / eval_games,
135             'draw_rate': draws / eval_games,
136             'loss_rate': losses / eval_games,
137             'epsilon': agent.epsilon,
138         })
139
140     return training_history

```

I Appendix – Tic Tac Toe Game Implementation

Listing 8: tic_tac_toe.py – Tic Tac Toe game engine

```

1  import numpy as np
2  import copy
3  from games.base_game import BaseGame
4
5  class TicTacToe(BaseGame):
6      def __init__(self):
7          self.board = np.zeros(9, dtype=int)
8          self.current_player = 1
9          self.done = False
10         self.winner = None
11
12         def reset(self) -> np.ndarray:
13             self.board = np.zeros(9, dtype=int)
14             self.current_player = 1
15             self.done = False

```

```

16         self.winner = None
17         return self.board.copy()
18
19     def get_valid_moves(self) -> list:
20         if self.done:
21             return []
22         return [i for i in range(9) if self.board[i] == 0]
23
24     def make_move(self, move: int, player: int) -> tuple:
25         if self.done:
26             return self.board.copy(), 0, True, {'winner': self.winner}
27         if self.board[move] != 0:
28             return self.board.copy(), -1, True, \
29                 {'winner': -player, 'invalid': True}
30         self.board[move] = player
31         winner = self.check_winner()
32         if winner is not None:
33             self.done = True
34             self.winner = winner
35             if winner == player:
36                 reward = 1.0
37             elif winner == 0:
38                 reward = 0.3
39             else:
40                 reward = -1.0
41             return self.board.copy(), reward, True, \
42                 {'winner': winner}
43         self.current_player = -player
44         return self.board.copy(), 0.0, False, {'winner': None}
45
46     def check_winner(self) -> int:
47         b = self.board.reshape(3, 3)
48         # Rows
49         for row in b:
50             if abs(row.sum()) == 3:
51                 return row[0]
52         # Columns
53         for col in b.T:
54             if abs(col.sum()) == 3:
55                 return col[0]
56         # Diagonals
57         d1 = b[0, 0] + b[1, 1] + b[2, 2]
58         d2 = b[0, 2] + b[1, 1] + b[2, 0]
59         if abs(d1) == 3:
60             return b[1, 1]
61         if abs(d2) == 3:
62             return b[1, 1]
63         # Draw
64         if 0 not in self.board:
65             return 0
66         return None
67
68     def clone(self):
69         g = TicTacToe()
70         g.board = self.board.copy()
71         g.current_player = self.current_player
72         g.done = self.done

```

```

73         g.winner = self.winner
74         return g
75
76     def render(self) -> str:
77         symbols = {0: '.', 1: 'X', -1: 'O'}
78         b = self.board.reshape(3, 3)
79         rows = []
80         for row in b:
81             rows.append(' | '.join(symbols[v] for v in row))
82         sep = '-' * 9
83         return f'\n{sep}\n'.join(rows)
84
85     def get_state_key(self) -> str:
86         return str(tuple(self.board))

```

J Appendix – Connect 4 Game Implementation

Listing 9: connect4.py – Connect 4 game engine with gravity mechanic

```

1  import numpy as np
2  from games.base_game import BaseGame
3
4  class Connect4(BaseGame):
5      ROWS = 6
6      COLS = 7
7
8      def __init__(self):
9          self.board = np.zeros((self.ROWS, self.COLS), dtype=int)
10         self.current_player = 1
11         self.done = False
12         self.winner = None
13
14     def reset(self) -> np.ndarray:
15         self.board = np.zeros((self.ROWS, self.COLS), dtype=int)
16         self.current_player = 1
17         self.done = False
18         self.winner = None
19         return self.board.copy()
20
21     def get_valid_moves(self) -> list:
22         if self.done:
23             return []
24         return [c for c in range(self.COLS)
25                 if self.board[0, c] == 0]
26
27     def make_move(self, move: int, player: int) -> tuple:
28         if self.done:
29             return self.board.copy(), 0, True, \
30                 {'winner': self.winner}
31         if move not in self.get_valid_moves():
32             return self.board.copy(), -1, True, \
33                 {'winner': -player, 'invalid': True}
34
35         # Gravity: drop piece to lowest empty row
36         row = None
37         for r in range(self.ROWS - 1, -1, -1):

```

```

38         if self.board[r, move] == 0:
39             row = r
40             break
41     self.board[row, move] = player
42
43     winner = self.check_winner()
44     if winner is not None:
45         self.done = True
46         self.winner = winner
47         if winner == player:
48             reward = 1.0
49         elif winner == 0:
50             reward = 0.3
51         else:
52             reward = -1.0
53         return self.board.copy(), reward, True, \
54             {'winner': winner}
55     self.current_player = -player
56     return self.board.copy(), 0.0, False, \
57         {'winner': None}
58
59     def check_winner(self) -> int:
60         b = self.board
61         # Horizontal
62         for r in range(self.ROWS):
63             for c in range(self.COLS - 3):
64                 window = b[r, c:c+4]
65                 if abs(window.sum()) == 4 \
66                     and 0 not in window:
67                     return b[r, c]
68         # Vertical
69         for r in range(self.ROWS - 3):
70             for c in range(self.COLS):
71                 window = b[r:r+4, c]
72                 if abs(window.sum()) == 4 \
73                     and 0 not in window:
74                     return b[r, c]
75         # Positive diagonal
76         for r in range(self.ROWS - 3):
77             for c in range(self.COLS - 3):
78                 window = [b[r+i, c+i] for i in range(4)]
79                 if abs(sum(window)) == 4 \
80                     and 0 not in window:
81                     return window[0]
82         # Negative diagonal
83         for r in range(self.ROWS - 3):
84             for c in range(3, self.COLS):
85                 window = [b[r+i, c-i] for i in range(4)]
86                 if abs(sum(window)) == 4 \
87                     and 0 not in window:
88                     return window[0]
89         # Draw
90         if len(self.get_valid_moves()) == 0:
91             return 0
92         return None
93
94     def clone(self):
95         g = Connect4()

```

```

96         g.board = self.board.copy()
97         g.current_player = self.current_player
98         g.done = self.done
99         g.winner = self.winner
100         return g
101
102     def render(self) -> str:
103         symbols = {0: '.', 1: 'X', -1: 'O'}
104         rows = []
105         for r in range(self.ROWS):
106             rows.append(' '.join(
107                 symbols[v] for v in self.board[r]))
108         header = ' '.join(
109             str(c) for c in range(self.COLS))
110         return header + '\n' + '\n'.join(rows)
111
112     def get_state_key(self) -> str:
113         return str(tuple(self.board.flatten()))

```

K Appendix – Entry Point

Listing 10: main.py – CLI entry point for training, tournaments, and interactive play

```

1  import argparse
2  import os
3  import sys
4  import random
5  import numpy as np
6
7
8  def set_seed(seed: int = 42):
9      random.seed(seed)
10     np.random.seed(seed)
11     try:
12         import torch
13         torch.manual_seed(seed)
14     except ImportError:
15         pass
16
17
18  def get_game(game_name: str):
19     from games.tic_tac_toe import TicTacToe
20     from games.connect4 import Connect4
21     if game_name == 'ttt':
22         return TicTacToe
23     elif game_name == 'c4':
24         return Connect4
25     else:
26         raise ValueError(f"Unknown game: {game_name}")
27
28
29  def get_agent(agent_name: str, player: int,
30               depth: int = None, game_name: str = 'ttt'):
31     from agents.random_agent import RandomAgent
32     from agents.default_opponent import DefaultOpponent
33     from agents.minimax_agent import MiniMaxAgent

```

```

34     from agents.alphabeta_agent import AlphaBetaAgent
35     from agents.qlearning_agent import QLearningAgent
36     from agents.dqn_agent import DQNAgent
37
38     input_size = 9 if game_name == 'ttt' else 42
39     output_size = 9 if game_name == 'ttt' else 7
40
41     agents = {
42         'random': lambda: RandomAgent(player, 'Random'),
43         'default': lambda: DefaultOpponent(
44             player, 'Default'),
45         'minimax': lambda: MiniMaxAgent(
46             player, 'MiniMax',
47             max_depth=depth),
48         'alphabeta': lambda: AlphaBetaAgent(
49             player, 'AlphaBeta',
50             max_depth=depth),
51         'qlearning': lambda: QLearningAgent(
52             player, 'QLearning'),
53         'dqn': lambda: DQNAgent(
54             player, 'DQN',
55             input_size=input_size,
56             output_size=output_size),
57     }
58     if agent_name not in agents:
59         raise ValueError(f"Unknown agent: {agent_name}")
60     return agents[agent_name]()
61
62
63 def mode_play(args):
64     depth = args.depth if hasattr(args, 'depth') else None
65     agent1 = get_agent(args.agent1, 1, depth=depth,
66                        game_name=args.game)
67     agent2 = get_agent(args.agent2, -1, depth=depth,
68                        game_name=args.game)
69
70     game_class = get_game(args.game)
71     game = game_class()
72     game.reset()
73     print(f"\n=== {agent1.name} (X) vs "
74           f"{agent2.name} (O) ===")
75     print(game.render())
76
77     current = agent1
78     move_num = 0
79     while True:
80         move = current.get_move(game)
81         _, _, done, info = game.make_move(
82             move, current.player)
83         move_num += 1
84         print(f"Move {move_num}: "
85               f"{current.name} plays {move}")
86         print(game.render())
87         if done:
88             winner = info.get('winner')
89             if winner == 1:
90                 print(f"Winner: {agent1.name} (X)")
91             elif winner == -1:

```

```

92         print(f"Winner: {agent2.name} (0)")
93     else:
94         print("Result: Draw")
95         break
96     current = agent2 if current is agent1 \
97         else agent1
98
99
100 def mode_train(args):
101     from experiments.runner import train_rl_agent
102     from agents.default_opponent import DefaultOpponent
103     from agents.random_agent import RandomAgent
104
105     game_class = get_game(args.game)
106     episodes = args.episodes
107     game_label = args.game.upper()
108     input_size = 9 if args.game == 'ttt' else 42
109     output_size = 9 if args.game == 'ttt' else 7
110
111     if args.agent == 'qlearning':
112         from agents.qlearning_agent import QLearningAgent
113         agent = QLearningAgent(1, 'QLearning')
114     elif args.agent == 'dqn':
115         from agents.dqn_agent import DQNAgent
116         agent = DQNAgent(1, 'DQN',
117                           input_size=input_size,
118                           output_size=output_size)
119     else:
120         print(f"Agent '{args.agent}' does not "
121               f"require training.")
122         return
123
124     if args.game == 'c4':
125         opponent = RandomAgent(-1, 'Random')
126     else:
127         opponent = DefaultOpponent(-1, 'Default')
128
129     print(f"\nTraining {agent.name} on {game_label} "
130           f"for {episodes} episodes "
131           f"vs {opponent.name}...")
132     history = train_rl_agent(
133         agent, opponent, game_class,
134         num_episodes=episodes)
135
136     os.makedirs('models', exist_ok=True)
137     ext = '.pkl' if args.agent == 'qlearning' else '.pt'
138     model_path = os.path.join(
139         'models', f'{args.agent}_{args.game}{ext}')
140     agent.save(model_path)
141     print(f"Model saved to {model_path}")
142
143     if history:
144         final = history[-1]
145         print(f"\nFinal evaluation "
146               f"(episode {final['episode']}):")
147         print(f"  Win rate: {final['win_rate']:.2%}")
148         print(f"  Draw rate: {final['draw_rate']:.2%}")
149         print(f"  Loss rate: {final['loss_rate']:.2%}")

```

```

150
151
152 def mode_tournament(args):
153     from experiments.runner import run_tournament
154     game_class = get_game(args.game)
155     depth = args.depth if hasattr(args, 'depth') \
156         else None
157     agent1 = get_agent(
158         args.agent1 if hasattr(args, 'agent1')
159         else 'alphabeta',
160         1, depth=depth, game_name=args.game)
161     agent2 = get_agent(
162         args.agent2 if hasattr(args, 'agent2')
163         else 'default',
164         -1, depth=depth, game_name=args.game)
165     num_games = args.num_games
166
167     print(f"\n=== Tournament: {agent1.name} vs "
168           f"{agent2.name} ({args.game.upper()}) ===")
169     results = run_tournament(
170         agent1, agent2, game_class,
171         num_games=num_games)
172
173     print(f"\nResults over {num_games} games:")
174     print(f"    {agent1.name} wins: "
175           f"{results['agent1_wins']} "
176           f"({results['agent1_win_rate']:.2%})")
177     print(f"    {agent2.name} wins: "
178           f"{results['agent2_wins']} "
179           f"({results['agent2_win_rate']:.2%})")
180     print(f"    Draws: {results['draws']} "
181           f"({results['draw_rate']:.2%})")
182     print(f"    Avg game length: "
183           f"{results['avg_game_length']:.1f} moves")
184
185
186 def mode_interactive(args):
187     depth = args.depth if hasattr(args, 'depth') else 5
188     ai = get_agent(
189         args.opponent if hasattr(args, 'opponent')
190         else 'minimax',
191         -1, depth=depth, game_name=args.game)
192
193     game_class = get_game(args.game)
194     game = game_class()
195     game.reset()
196     print(f"\n=== Interactive Mode: "
197           f"You (X) vs {ai.name} (O) ===")
198     print(game.render())
199
200     current_player = 1
201     while True:
202         if current_player == 1:
203             valid = game.get_valid_moves()
204             while True:
205                 try:
206                     move = int(
207                         input(f"Your move {valid}: "))

```

```

208             if move in valid:
209                 break
210             print("Invalid move, try again.")
211         except (ValueError, KeyboardInterrupt):
212             print("\nGame aborted.")
213             return
214     else:
215         move = ai.get_move(game)
216         print(f"{ai.name} plays: {move}")
217
218     _, _, done, info = game.make_move(
219         move, current_player)
220     print(game.render())
221     if done:
222         winner = info.get('winner')
223         if winner == 1:
224             print("You win!")
225         elif winner == -1:
226             print(f"{ai.name} wins!")
227         else:
228             print("Draw!")
229         break
230     current_player = -current_player
231
232
233 def main():
234     parser = argparse.ArgumentParser(
235         description='AI Game Agents')
236     parser.add_argument(
237         '--mode',
238         choices=['play', 'train', 'tournament',
239                 'interactive'],
240         default='play')
241     parser.add_argument(
242         '--game', choices=['ttt', 'c4'], default='ttt')
243     parser.add_argument('--agent1', default='minimax')
244     parser.add_argument('--agent2', default='default')
245     parser.add_argument('--agent', default='qlearning')
246     parser.add_argument('--opponent', default='minimax')
247     parser.add_argument('--depth', type=int, default=None)
248     parser.add_argument(
249         '--episodes', type=int, default=50000)
250     parser.add_argument(
251         '--num-games', type=int, default=1000,
252         dest='num_games')
253     parser.add_argument('--seed', type=int, default=42)
254
255     args = parser.parse_args()
256     set_seed(args.seed)
257
258     if args.mode == 'play':
259         mode_play(args)
260     elif args.mode == 'train':
261         mode_train(args)
262     elif args.mode == 'tournament':
263         mode_tournament(args)
264     elif args.mode == 'interactive':
265         mode_interactive(args)

```

```
266
267
268 if __name__ == '__main__':
269     main()
```